ITERATORS IN PYTHON

Iterable objects, iter(), next(), and the iteration process

Learning objective Understand how Python retrieves elements one at a time, how an iterator keeps track of its position, and how a for loop uses the iterator mechanism.

1. What Is an Iterable?

An iterable is an object that allows Python to access its elements one at a time. Lists, tuples, strings, dictionaries, sets and ranges are common examples.

```
numbers = [10, 20, 30, 40]

for number in numbers:
    print(number)
```

In this example, the list numbers is an iterable. The for loop accesses each element one by one.

Key point An iterable is an object from which Python can obtain an iterator.

2. What Is an Iterator?

Definition An iterator allows Python to retrieve elements one at a time while keeping track of where it has reached.

An iterator keeps track of its current position. Each time Python requests the next element, the iterator moves forward and provides that element.

Think of a bookmark in a book: it marks where you stopped so you can continue from the next position.

3. Difference Between an Iterable and an Iterator

Aspect	Iterable	Iterator
Meaning	An object from which an iterator can be obtained.	An object that allows Python to retrieve elements one at a time while keeping track of its position.
Purpose	Provides a source of elements.	Keeps track of progress and provides the next element when requested.
Examples	List, tuple, string, set, dictionary, range.	list_iterator, tuple_iterator, str_iterator, range_iterator.
Common operation	iter() obtains an iterator from it.	next() retrieves the next element.

4. Creating an Iterator Using iter()

Python provides the built-in iter() function to obtain an iterator from an iterable.

```
numbers = [10, 20, 30, 40]

number_iterator = iter(numbers)

print(type(numbers))
print(type(number_iterator))
```

Typical output:

```
<class 'list'>
<class 'list_iterator'>
```

numbers refers to the original list, while number_iterator refers to a separate iterator object created from that list.

Important Creating an iterator does not convert the original list into an iterator. The list and the iterator are separate objects.

5. Retrieving Elements Using next()

The next() function retrieves the next element from an iterator. Each call moves the iterator forward by one position.

```
numbers = [10, 20, 30, 40]

number_iterator = iter(numbers)

print(next(number_iterator))
print(next(number_iterator))
print(next(number_iterator))
```

Output:

```
10
20
30
```

The iterator remembers its position after each call. Therefore, the second call returns 20 rather than returning 10 again.

6. How Does an Iterator Keep Track of Its Position?

Consider the following list and its iterator:

```
numbers = [10, 20, 30, 40, 50]
number_iterator = iter(numbers)
```

Each call to next() retrieves the element at the current position and advances the iterator to the next position.

Call to next()	Element returned	Iterator moves to
First call	10	20
Second call	20	30
Third call	30	40
Fourth call	40	50
Fifth call	50	No remaining element

The iterator maintains this position internally. We do not need to create and update a separate index variable.

7. What Happens When an Iterator Is Exhausted?

When an iterator has provided all its elements, it is called an exhausted iterator. Calling next() again raises an exception named StopIteration.

```
numbers = [10, 20]
```

```
number_iterator = iter(numbers)

print(next(number_iterator))
print(next(number_iterator))
print(next(number_iterator))  # Raises StopIteration
```

Output before the exception:

```
10
20
```

The third call to next() raises StopIteration because the iterator has no more elements to provide.

8. Handling StopIteration

We can use try and except to handle StopIteration and display a message when there are no more elements.

```
numbers = [10, 20]
number_iterator = iter(numbers)

try:
    print(next(number_iterator))
    print(next(number_iterator))
    print(next(number_iterator))
except StopIteration:
    print("No more elements available")
```

Output:

```
10
20
No more elements available
```

Remember An exhausted iterator does not automatically restart. To iterate again, create a new iterator from the original iterable.

9. Does Using an Iterator Change the Original List?

No. The iterator keeps track of its own position. Retrieving elements using next() does not remove elements from the original list.

```
numbers = [10, 20, 30]

number_iterator = iter(numbers)

print(next(number_iterator))
print(numbers)
```

Output:

```
10
[10, 20, 30]
```

The iterator has moved past the first element, but the original list still contains all three elements.

10. How Does a for Loop Use an Iterator?

A for loop uses the iterator mechanism automatically. It obtains an iterator from the iterable, retrieves elements one at a time, and stops when there are no more elements.

```
numbers = [10, 20, 30]

for number in numbers:
    print(number)
```

Conceptually, the for loop follows these steps:

- Obtains an iterator from the iterable.
- Requests the next element from the iterator.
- Executes the loop body using that element.
- Repeats the process until the iterator raises StopIteration.
- Stops automatically; we do not need to handle StopIteration ourselves.

Practical guidance Use a for loop for everyday iteration. Understanding iter() and next() helps explain how the loop works internally and gives you manual control when required.

11. Example: Retrieving Student Names

This example uses an iterator to retrieve student names one at a time.

```
students = ["Anu", "Bala", "Charan", "Divya"]

student_iterator = iter(students)

print("First student:", next(student_iterator))
print("Second student:", next(student_iterator))
print("Third student:", next(student_iterator))
```

Output:

```
First student: Anu
Second student: Bala
Third student: Charan
```

The iterator is now positioned to return Divya on the next call to next().

12. A Brief Look at Custom Iterators

Python also allows us to create our own iterator classes. A custom iterator generally implements two special methods:

- __iter__() – returns the iterator object itself.
- __next__() – returns the next element. When no elements remain, it raises StopIteration.

Custom iterator classes are useful when we need to define our own sequence or control how elements are produced. They are an additional topic; they are not required for using built-in iterators such as list iterators.

For now Focus on iterable, iterator, iter(), next(), and StopIteration. These are the essential concepts for understanding iteration.

13. Practice Exercises

Exercise 1 – List iterator: Create a list containing five city names. Use iter() to create an iterator and next() to print the first three city names.

Exercise 2 – Iterator position: Create an iterator from a list of five numbers. Call next() twice. Then call next() again and identify which element is returned.

Exercise 3 – StopIteration: Create an iterator from a list containing two elements. Call next() three times. Observe the exception raised by the third call.

Exercise 4 – Handle exhaustion: Repeat Exercise 3, but use try and except to handle StopIteration and display a suitable message.

Exercise 5 – Original list: Create an iterator from a list. Retrieve two elements using next(). Print the original list and verify whether its contents have changed.

Exercise 6 – Manual iteration and for loop: Print all elements of a list using a for loop. Then create a new iterator and print the same elements using next() calls.

Quick Recap

Iterable An object from which Python can obtain an iterator.

Iterator An iterator allows Python to retrieve elements one at a time while keeping track of where it has reached.

iter() Obtains an iterator from an iterable.

next() Retrieves the next element from an iterator.

StopIteration Indicates that an iterator has no more elements.

for loop Uses the iterator mechanism automatically and handles the end of iteration.